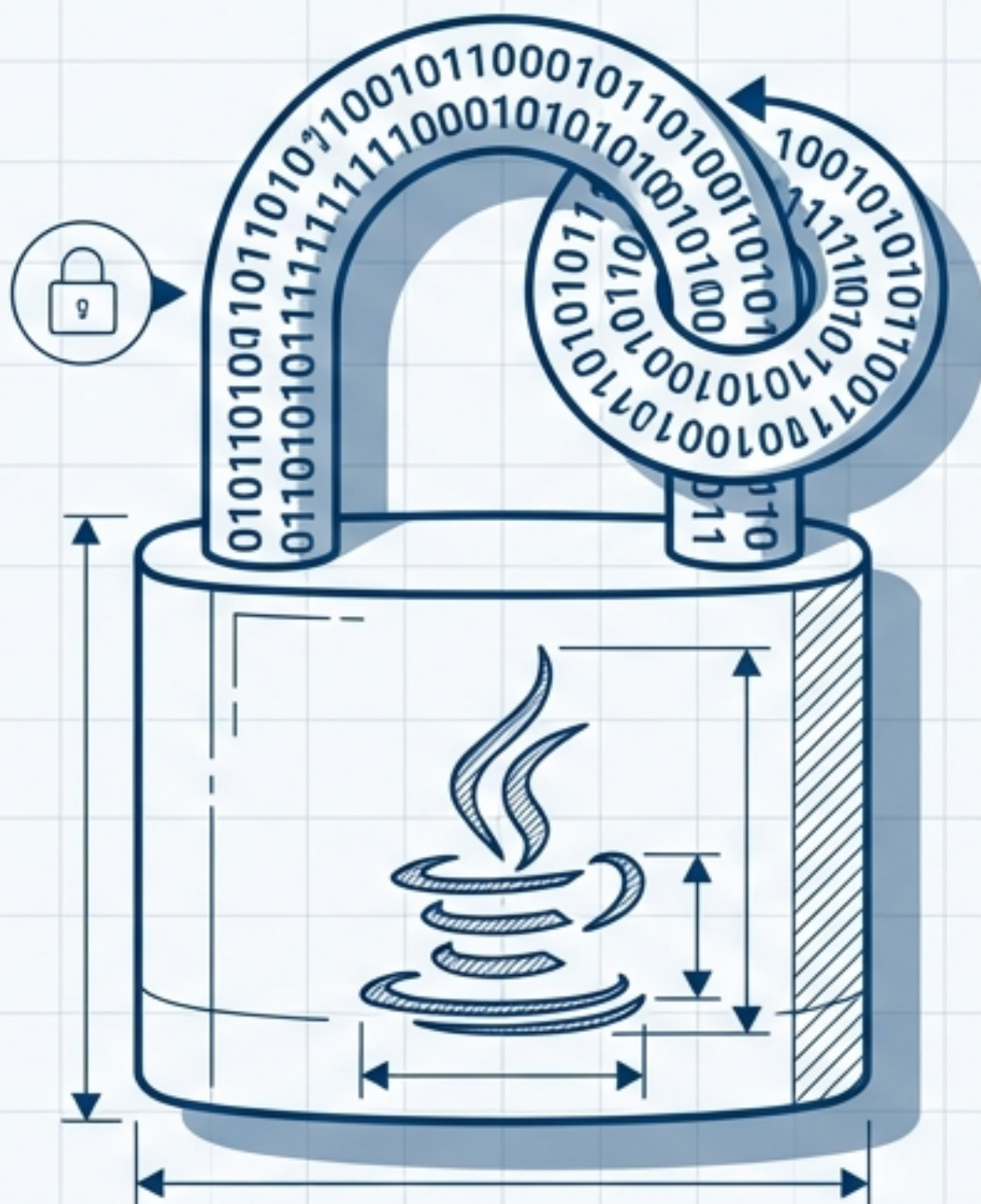


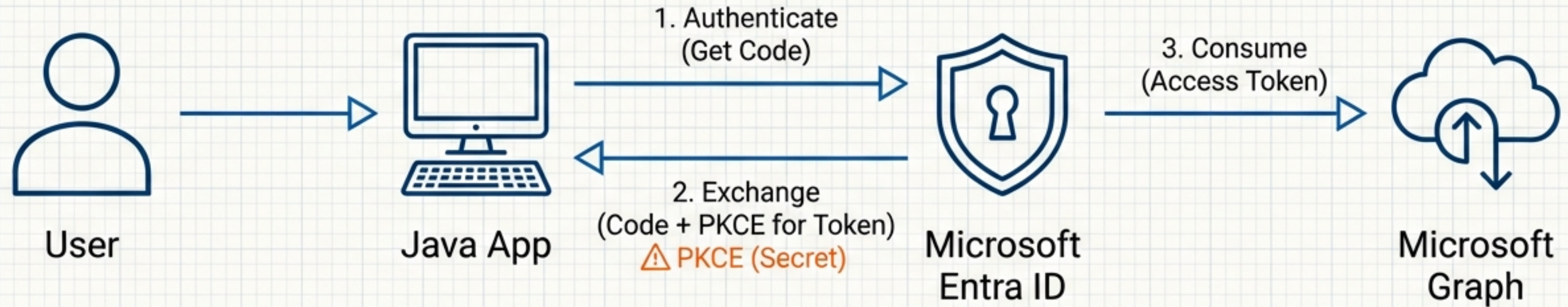


Secure Authentication for Java Desktop Apps

A Builder's Guide to OAuth 2.0, PKCE, and Microsoft Entra ID Integration

TARGET: PUBLIC CLIENT APPLICATIONS (WINDOWS / LINUX / MACOS)

The Architecture of a Secure Flow



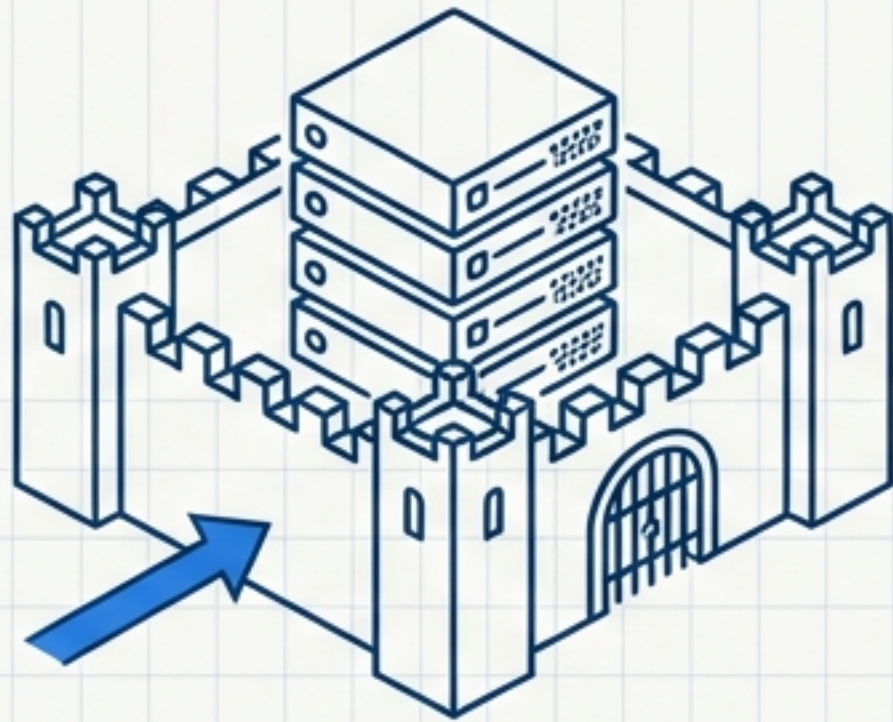
The User logs in via the browser. The App captures the Authorization Code.

The App proves its identity using the PKCE secret and exchanges the code for tokens.

The App presents the Access Token to fetch user data or API resources.

Why Desktop Apps Cannot Keep Secrets

Confidential Client (Server Web App)



Code runs on a secure backend.
Can safely store a "client_secret".
The end user never sees the code.

Public Client (Native/Desktop)



Code runs on the user's device.
Can be decompiled. Traffic can be inspected.

"Public clients... must not use secrets."
- OAuth 2.0 Spec

The Solution: PKCE (Proof Key for Code Exchange)

Instead of a static password (client_secret), we generate a dynamic, cryptographic one-time secret for every single login attempt.

Blueprinting the App Registration

Azure Entra admin center

Register an application

Name

Supported account types

☒ Accounts in this organizational directory only (Single tenant)

☐ Accounts in theses

Application (client) ID

e.g., 8888-9999-aaaa-bbbb

The Public Identifier.

Platform configuration

☒ Mobile and desktop applications

Platform configuration

✓ Mobile and desktop applications

CRITICAL: Do not choose "Web" or "SPA".

Redirect URIs

Suggested redirect URIs for public clients (mobile, desktop)

http://localhost

Redirect URIs

http://localhost

Loopback for system browser support.

API permissions

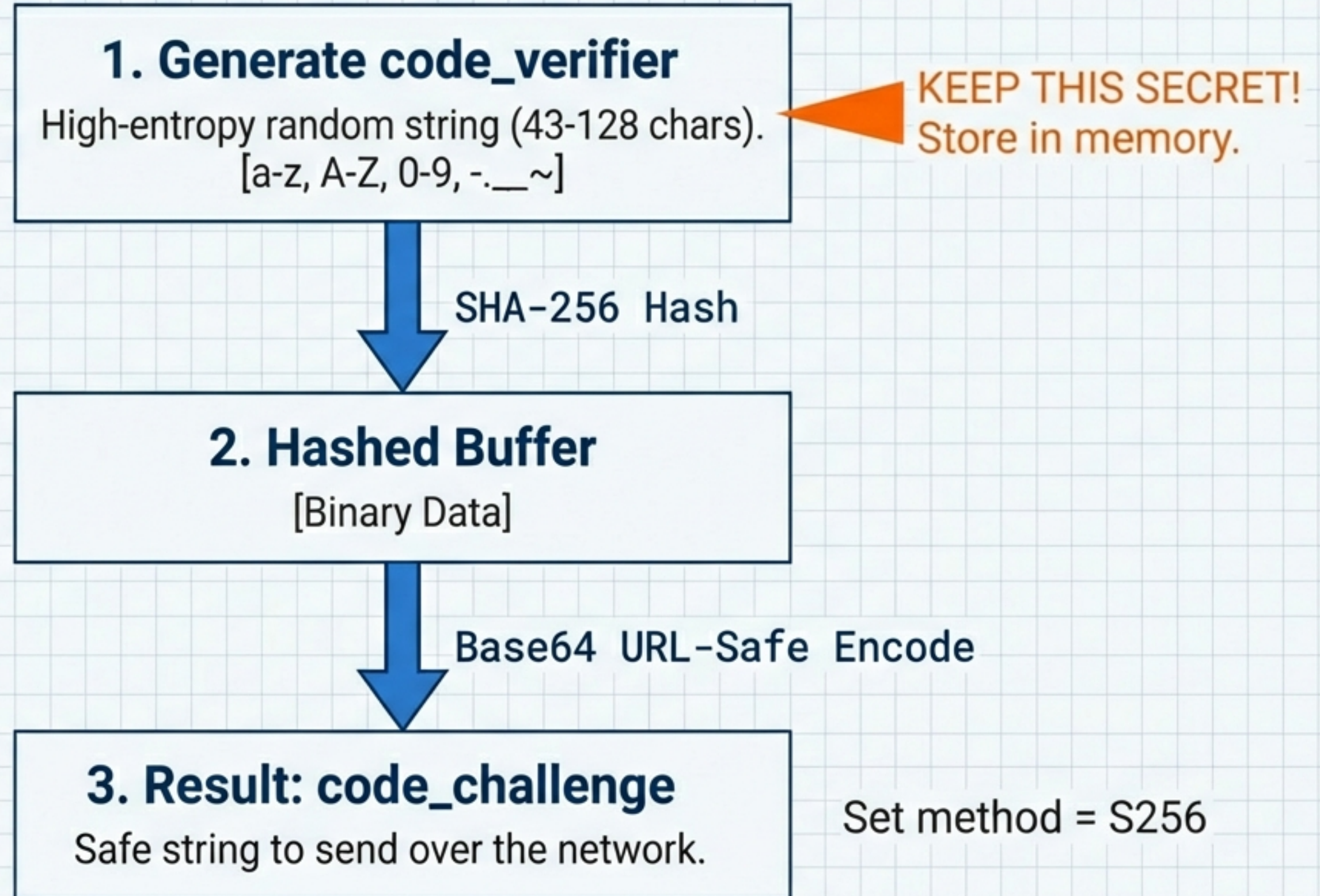
API permissions

☒ User.Read (Delegated)

User.Read (Delegated) description, an automatinptw and opais to User.Read.

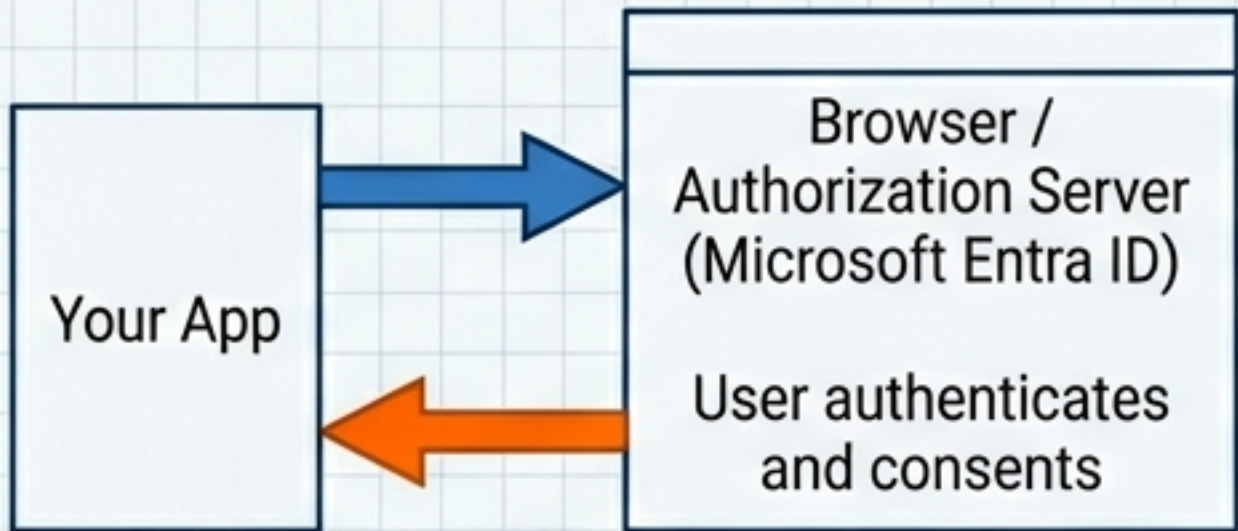
Access on behalf of the user.

Step 1: Generating the Proof Key (PKCE)



Step 2: The Authorization Request

GET <https://login.microsoftonline.com/{tenant}/oauth2/v2.0/authorize>

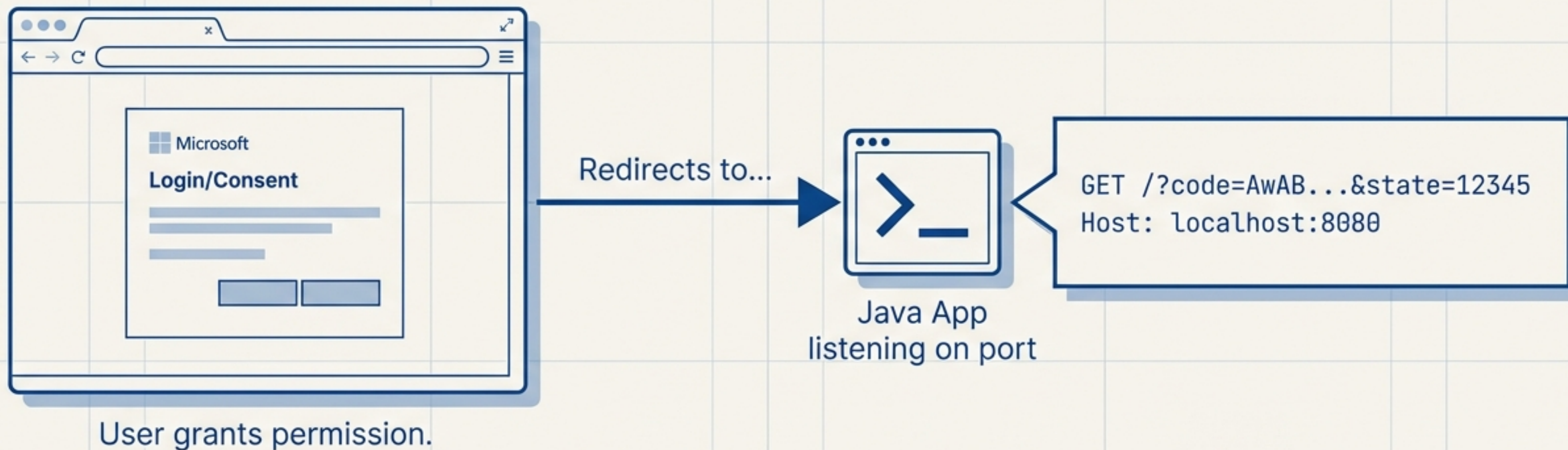


Authorization Code
(Temporary, one-time use)

Critical security
parameter from Step 1

Parameter	Value / Description
client_id	Your Application ID
response_type	code (Triggers Auth Code Flow)
redirect_uri	http://localhost:{port} (URL Encoded)
scope	openid offline_access https://graph.microsoft.com/User.Read
code_challenge	{Base64 String from Step 1}
code_challenge_method	S256
state	{Random String} (Anti-CSRF protection)

Step 3: User Consent & The Loopback Redirect

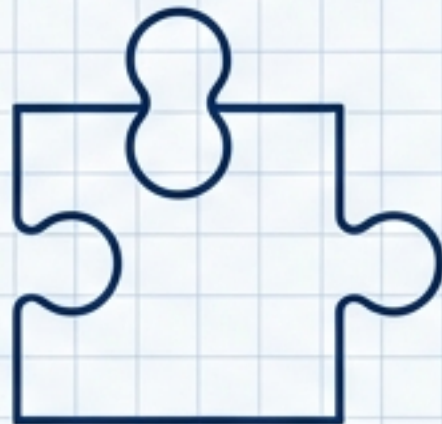
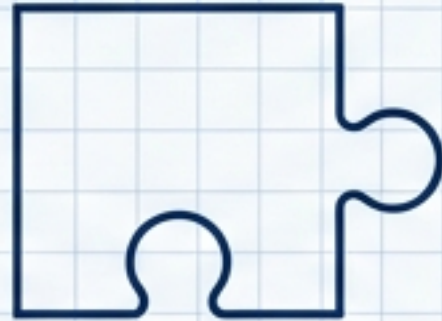


The App captures the URL. It validates the 'state' matches the request, and extracts the 'code'. The code expires in ~60 seconds.

Step 4: Redeeming the Code for Tokens

Puzzle Match

code_challenge
(Sent in Step 2)

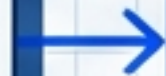


code_verifier
(Sent Now)



Entra ID

Hashes the verifier.
If it matches the
challenge,



Token Issued



it issues the token.

HTTP Request

```
POST /{tenant}/oauth2/v2.0/token
```

```
grant_type: authorization_code
```

```
code: {The code from Step 3}
```

```
redirect_uri: http://localhost:{port}
```

```
client_id: {Your App ID}
```

```
code_verifier: {Original Un-Hashed String}
```


Anatomy of the Token Response



The Key (Short-lived)

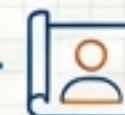
Used to call the API.
Short-lived (1 hour).

```
{  
  "access_token": "eyJ0eXAiOi...",  
  "token_type": "Bearer",  
  "expires_in": 3599,  
  "scope": "User.Read",  
  "refresh_token": "AwABAAAA...",  
  "id_token": "eyJ0eXAiOi..."  
}
```



The Session (Long-lived)

Used to get new
access tokens.
Long-lived.



The Identity

Contains user info
(Name, Email).
Requires 'openid' scope.



Expiry

Seconds until expiry.

Step 5: Calling Microsoft Graph

Retrieving User Profile Data



The Request Card

HTTP Request

GET https://graph.microsoft.com/v1.0/me

Headers:

Authorization: Bearer {access_token}

Content-Type: application/json



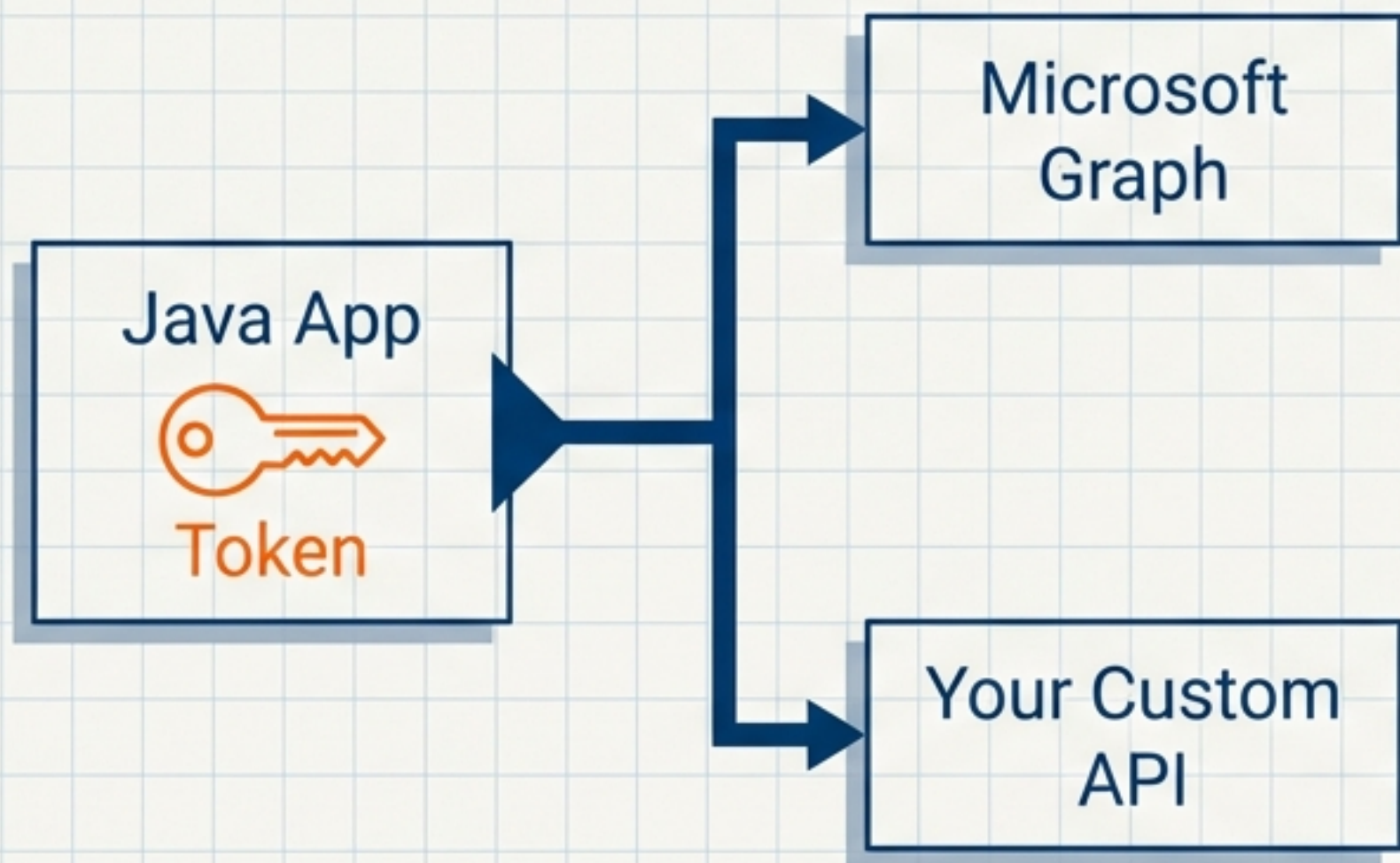
The Response Card

200 OK

```
{
  "displayName": "Alex Wilber",
  "jobTitle": "Developer",
  "mail": "alex@contoso.com",
  "userPrincipalName": "alex@contoso.com"
}
```

Requires scope 'User.Read' in the access token.

Step 6: Connecting to Custom REST APIs



The flow is identical for your own services. The API validates the token against the Entra ID tenant.

Header:

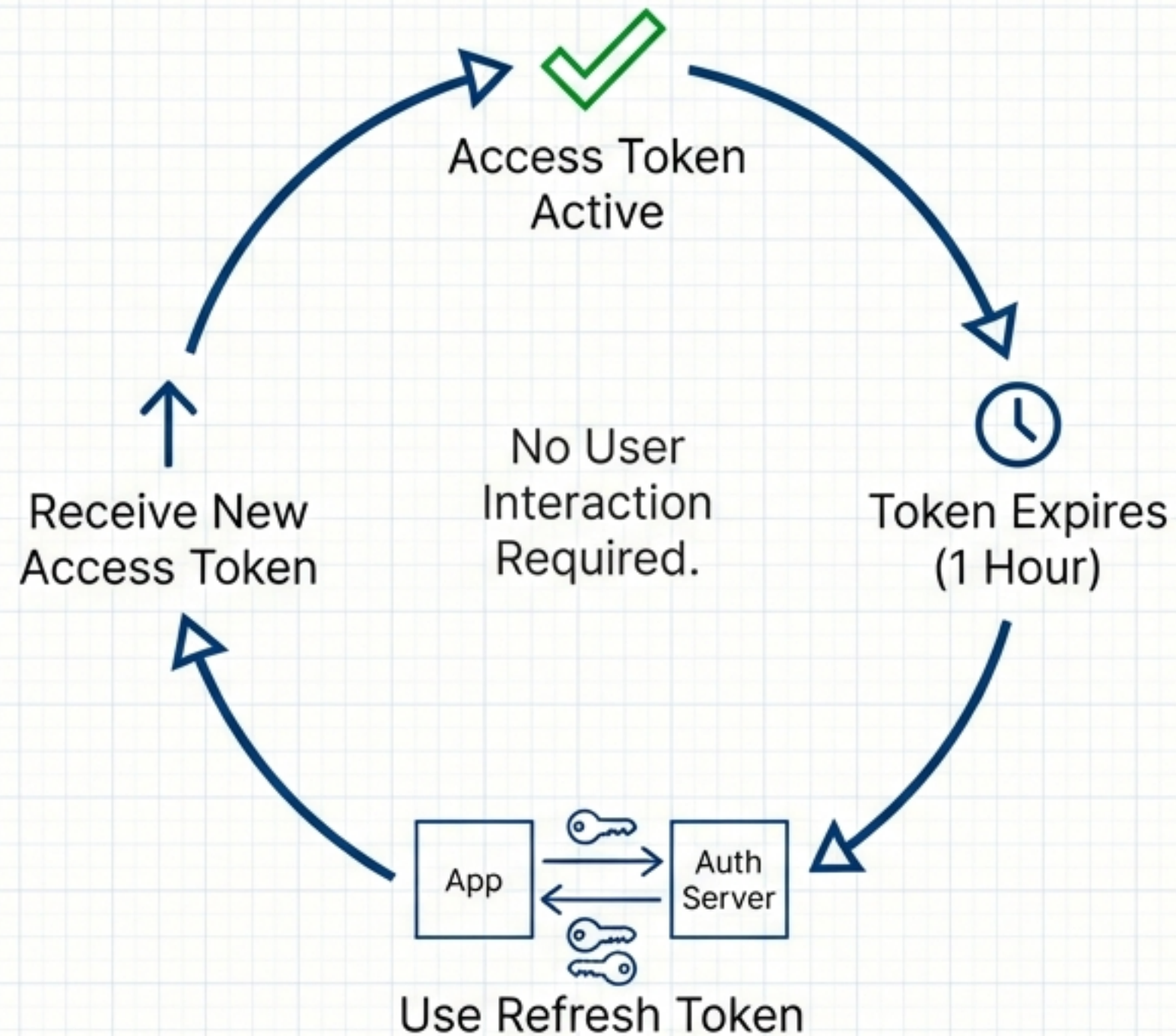
```
Authorization: Bearer {access_token}
```

Implementation Details:

- ≡ - Reuse token if audiences match.
- ≡ - Or use Refresh Token to request a new Access Token for 'api://my-company-app/.default'. (Scope JetBrains Mono)

Maintaining the Session

Maintaining Session (#003366) Helvetica Now Display Bold.
Technical Grey (#444444) Roboto, Reference slides.



The Request

```
POST /token
grant_type=refresh_token
refresh_token={stored_refresh_token}
scope={scopes}
```

result: New Access Token + New Refresh Token

Troubleshooting Common Errors

invalid_grant



Code verifier does not match challenge.

Fix:

- Ensure you send the original raw verifier, not re-hashed. Ensure specific encoding is correct.

Cross-Origin / CORS



Access blocked by CORS policy.

Fix:

- You are likely using a SPA Redirect URI in a non-SPA flow. Check App Registration platform settings.

invalid_scope



The scope is not configured.

Fix:

- Check resource permissions in Entra portal. Ensure use of fully qualified scopes.

interaction_required



Silent refresh failed (MFA, Password Change).

Fix:

- App must catch this and prompt user again with prompt=select_account.

Production Implementation: MSAL Java

Automating the Protocol

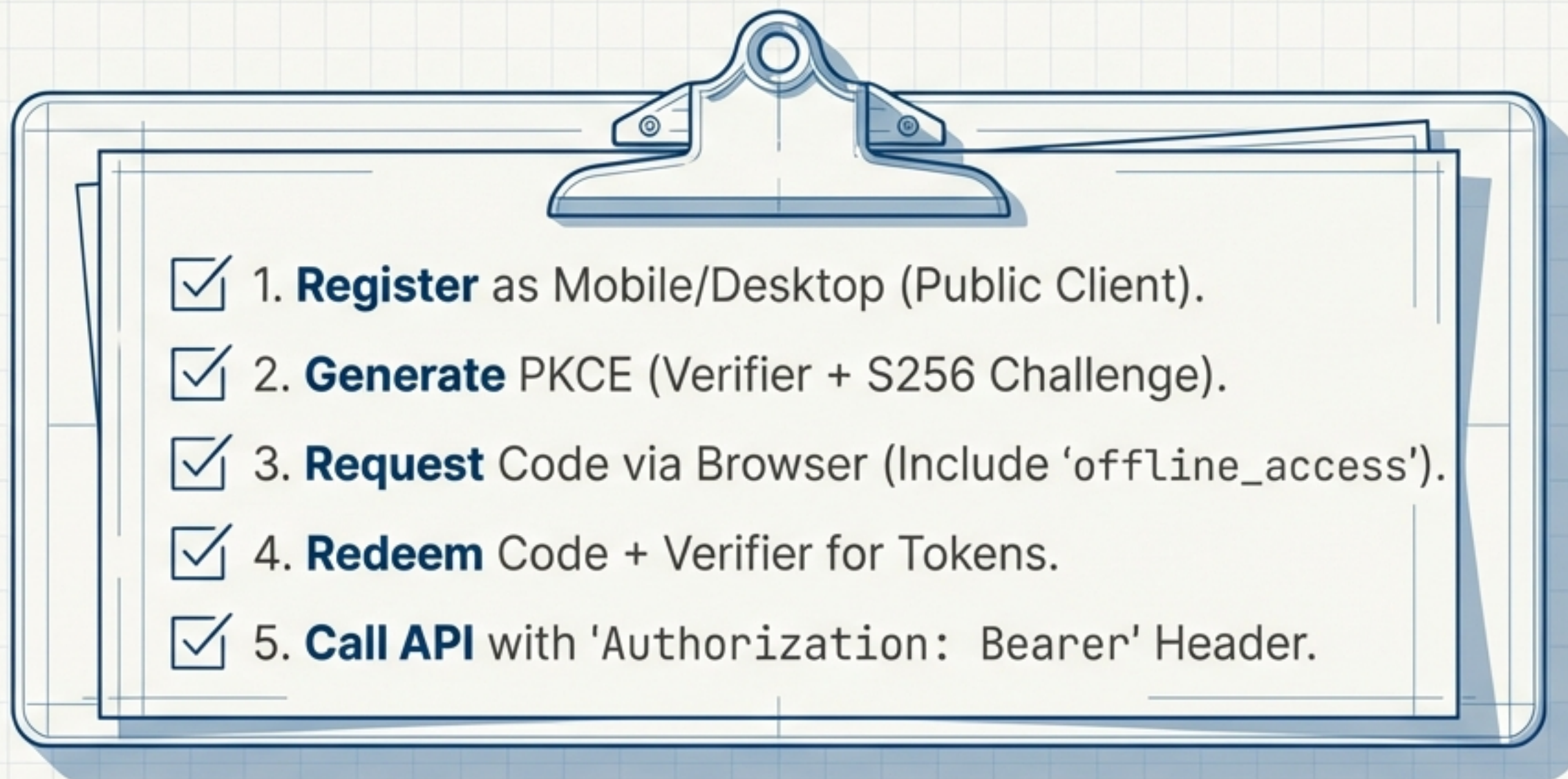
Do not manually implement HTTP flows in production. Use the Microsoft Authentication Library (MSAL).

- ☒ Automatic PKCE generation (Verifier & Challenge)
- ☒ Loopback listener management
- ☒ Secure Token Caching
- ☒ Silent Token Refresh Logic

Key Class Code Snippet

```
PublicClientApplication app =  
PublicClientApplication.builder(CLIENT_ID)  
    .authority(AUTHORITY)  
    .build();  
  
AcquireTokenByAuthorizationCodeParameters params =  
AcquireTokenByAuthorizationCodeParameters.builder(  
    authorizationCode,  
    new URI("http://localhost"))  
    .build();
```


The Builder's Checklist



Resources

- Library: MSAL Java ([GitHub/Maven](#))
- Testing: [Microsoft Graph Explorer](#)
- Spec: [RFC 7636](#) (PKCE)

Go build something secure.